

Shivani.S-192471030

Q66. Huffman vs Optimal BST (Frequency-Based Trees) Given frequencies [5,9,12,13,16,45], construct Huffman tree and Optimal BST. Compare structures. Analyze cost differences. Evaluate complexity.

Ans:

Q66. Huffman Tree vs Optimal BST

Frequency-Based Trees

```
import heapq
```

```
class HuffmanNode:
```

```
    def __init__(self, freq, key=None, left=None, right=None):
        self.freq = freq
        self.key = key
        self.left = left
        self.right = right
```

```
def build_huffman(keys, freq):
```

```
    heap = []
    counter = 0
```

```
    for i in range(len(keys)):
        node = HuffmanNode(freq[i], keys[i])
        heapq.heappush(heap, (freq[i], counter, node))
        counter += 1
```

```
    while len(heap) > 1:
        f1, c1, left = heapq.heappop(heap)
        f2, c2, right = heapq.heappop(heap)

        merged = HuffmanNode(f1 + f2, None, left, right)
        heapq.heappush(heap, (merged.freq, counter, merged))
        counter += 1
```

```
    return heap[0][2]
```

```

def generate_codes(root, code, codes):
    if root is None:
        return

    if root.key is not None:
        codes[root.key] = code
        return

    generate_codes(root.left, code + "0", codes)
    generate_codes(root.right, code + "1", codes)

```

```

def huffman_cost(freq, codes, keys):
    total = 0

    for i in range(len(keys)):
        total += freq[i] * len(codes[keys[i]])

    return total

```

```

def optimal_bst(keys, freq):
    n = len(keys)

    cost = [[0 for _ in range(n + 2)] for _ in range(n + 2)]
    root = [[0 for _ in range(n + 2)] for _ in range(n + 2)]

    prefix = [0]

    for f in freq:
        prefix.append(prefix[-1] + f)

    def sum_freq(i, j):
        return prefix[j] - prefix[i - 1]

    for i in range(1, n + 1):
        cost[i][i] = freq[i - 1]
        root[i][i] = i

```

```

for length in range(2, n + 1):
    for i in range(1, n - length + 2):
        j = i + length - 1
        cost[i][j] = float("inf")

        for r in range(i, j + 1):
            left_cost = cost[i][r - 1] if r > i else 0
            right_cost = cost[r + 1][j] if r < j else 0

            total_cost = left_cost + right_cost + sum_freq(i, j)

            if total_cost < cost[i][j]:
                cost[i][j] = total_cost
                root[i][j] = r

return cost, root

```

```

def print_root_matrix(root, n):
    for i in range(1, n + 1):
        for j in range(i, n + 1):
            print(root[i][j], end=" ")
        print()

```

```

keys = ["K1", "K2", "K3", "K4", "K5", "K6"]
freq = [5, 9, 12, 13, 16, 45]

```

```

# Huffman Tree
huffman_root = build_huffman(keys, freq)
codes = {}
generate_codes(huffman_root, "", codes)

print("Huffman Codes:")
for key in keys:
    print(key, ":", codes[key])

print("Huffman Cost:", huffman_cost(freq, codes, keys))

# Optimal BST

```

```

cost, root = optimal_bst(keys, freq)
n = len(keys)

print("\nOptimal BST Cost:", cost[1][n])

print("\nOBST Root Matrix:")
print_root_matrix(root, n)

print("\nComparison:")
print("Huffman tree minimizes weighted external path length.")
print("Optimal BST minimizes weighted search cost while preserving sorted key order.")
print("Huffman does not require sorted order, but OBST requires keys to remain ordered.")
print("Huffman Time Complexity: O(n log n)")
print("OBST Time Complexity: O(n^3)")

```

Q67. Word Break vs Word Wrap (Text DP Comparison)

Given string “iloveprogramming” and dictionary {“i”, “love”, “programming”}, solve Word Break. Also format words [3,2,2,5] with width=6 using Word Wrap. Compare DP outputs. Analyze differences.

Ans:

Q67. Word Break vs Word Wrap

Text DP Comparison

```
def word_break(s, dictionary):
```

```
    word_set = set(dictionary)
```

```
    dp = [False] * (len(s) + 1)
```

```
    previous = [-1] * (len(s) + 1)
```

```
    dp[0] = True
```

```
    for i in range(1, len(s) + 1):
```

```
        for j in range(i):
```

```
            if dp[j] and s[j:i] in word_set:
```

```
                dp[i] = True
```

```
                previous[i] = j
```

```
break
```

```
if not dp[len(s)]:
```

```
    return False, ""
```

```
    words = []
```

```
    index = len(s)
```

```
    while index > 0:
```

```
        prev_index = previous[index]
```

```
        words.append(s[prev_index:index])
```

```
        index = prev_index
```

```
    words.reverse()
```

```
    return True, " ".join(words)
```

```
def word_wrap(word_lengths, width):
```

```
    n = len(word_lengths)
```

```
    dp = [float("inf")] * (n + 1)
```

```
    next_break = [-1] * n
```

```
    dp[n] = 0
```

```
    for i in range(n - 1, -1, -1):
```

```
        line_length = 0
```

```
        for j in range(i, n):
```

```
            line_length += word_lengths[j]
```

```
            spaces_between_words = j - i
```

```
            total_length = line_length + spaces_between_words
```

```
        if total_length > width:
```

```
            break
```

```
        extra_spaces = width - total_length
```

```

if j == n - 1:
    cost = 0
else:
    cost = extra_spaces * extra_spaces + dp[j + 1]
    if cost < dp[i]:
        dp[i] = cost
        next_break[i] = j + 1
    lines = []
    i = 0
    while i < n:
        j = next_break[i]
        lines.append((i + 1, j))
        i = j
    return dp[0], lines

# Word Break Input
s = "iloveprogramming"
dictionary = ["i", "love", "programming"]
possible, segmentation = word_break(s, dictionary)
print("Word Break Result:")
print("Can be segmented:", possible)
print("Segmentation:", segmentation)

# Word Wrap Input
word_lengths = [3, 2, 2, 5]
width = 6
cost, lines = word_wrap(word_lengths, width)
print("\nWord Wrap Result:")
print("Minimum Cost:", cost)

```

```
print("Lines:")  
for line in lines:  
    print("Words", line[0], "to", line[1])  
print("\nComparison:")  
print("Word Break checks whether a string can be segmented using dictionary words.")  
print("Word Wrap arranges words into lines with minimum extra-space penalty.")  
print("Word Break output is True/False with segmentation.")  
print("Word Wrap output is minimum cost with line breaks.")
```